Projeto e Desenvolvimento de Sistema de Banco de Dados Relacional no Varejo Supermercadista

Antônio Gilson Oliveira Xavier Júnior, Fernanda Maria Neves Almeida.

Orientador(a)

Luciana Mara Freitas Diniz

PUC Minas

Curso de Tecnologia em Banco de Dados

Trabalho de desenvolvimento de Projeto e Desenvolvimento de Sistema de Banco de Dados Relacional no varejo supermercadista, apresentado à Pontifícia Universidade Católica de Minas Gerais como requisito parcial para obtenção do título de Tecnólogo em Banco de Dados.

Belo Horizonte

2025

SUMÁRIO

1. INTRODUÇÃO....3
2. ESPECIFICAÇÃO DO MINIMUNDO....4
3. REQUISITOS FUNCIONAIS....5
4. PROJETO CONCEITUAL....7
5. PROJETO LÓGICO....7
6. SOFTARES USADOS....11
7. SCRIPTS, CONSULTAS, VIEWS E RESULTADOS....13
8. CONCLUSÃO....33
9. REFERÊNCIAS BIBLIOGRÁFICAS....35

1. INTRODUÇÃO

O setor supermercadista no Brasil está em constante crescimento e passa por mudanças significativas. A forma de comprar dos clientes se transformou bastante: hoje, além das lojas físicas, eles usam aplicativos, sites e serviços de entrega. Para acompanhar essa evolução, os supermercados precisam de sistemas eficientes capazes de gerenciar um grande volume de informações, que vão desde o controle de estoque até o relacionamento com fornecedores e clientes. Nesse contexto, um banco de dados relacional bem estruturado se torna uma peça essencial para dar suporte a todas essas operações.

De acordo com a Associação Brasileira de Supermercados (ABRAS), o setor de varejo alimentar movimentou mais de R$ 1 trilhão em 2024, o que representa cerca de 9% do PIB nacional. Esse número não apenas mostra a importância econômica do setor, mas também evidência como uma gestão de dados eficiente é crucial para o desempenho das empresas e para o atendimento ao consumidor.

O objetivo deste trabalho é projetar e modelar um banco de dados relacional para supermercados. Ele será planejado para organizar informações sobre vendas, estoque, compras, logística, clientes, funcionários e fornecedores. Além disso, a estrutura proposta deve permitir integração com outros sistemas e facilitar a geração de relatórios para apoiar a tomada de decisão.

A relevância deste projeto está nos benefícios que um banco de dados bem estruturado pode oferecer: redução de perdas e falhas no controle de estoque, maior precisão e confiabilidade das informações, mais segurança para os dados e um melhor apoio a estratégias de vendas e fidelização de clientes. Estrategicamente, um bom banco de dados não só organiza a informação como também se torna uma ferramenta estratégica para aumentar a eficiência e a competitividade do supermercado.

2. ESPECIFICAÇÃO DO MINIMUNDO

Essa seção apresenta a descrição textual do minimundo do Sistema de Supermercado (v1.0), um banco de dados voltado para gerenciar o processo de vendas, cadastro de clientes, controle de produtos, fornecedores, estoque e funcionários em um ambiente de supermercado.

No supermercado, os clientes são identificados por seu CPF e possuem atributos como nome, endereço e telefone. Cada cliente pode realizar diversas compras, sendo que cada venda é obrigatoriamente vinculada a um cliente.

As vendas são registradas com um identificador próprio, data da venda e valor total. Uma venda é composta por itens de venda, que registram a quantidade de produtos adquiridos e o valor unitário de cada um deles.

Os itens de venda fazem referência obrigatória a um produto, sendo que um produto pode aparecer em vários itens de vendas diferentes.

Os produtos são identificados por um código único e possuem nome e preço unitário. Todo produto pertence a uma categoria, que é identificada por um código e possui um nome. Uma categoria pode ter diversos produtos associados, mas cada produto pertence a apenas uma categoria.

Além disso, cada produto possui um estoque, onde são registradas informações como a quantidade atual disponível e a quantidade mínima necessária para reposição.

Os produtos também estão relacionados a fornecedores, que são identificados por um código único e possuem atributos como nome, CNPJ, telefone e e-mail. Um fornecedor pode fornecer vários produtos ao supermercado, e cada produto pode estar vinculado a um ou mais fornecedores, dependendo da política de compras.

As vendas são realizadas por funcionários, identificados por um código único, CPF, nome e cargo. Cada venda obrigatoriamente deve estar associada a um funcionário responsável pelo atendimento.

Esse modelo permite organizar os principais processos do supermercado: o cadastro de clientes, a categorização e controle de produtos, o gerenciamento de

fornecedores, a administração do estoque, bem como o registro detalhado das vendas realizadas. Assim, torna-se possível rastrear quais clientes compraram, quando compraram, quais itens foram adquiridos, quais fornecedores forneceram os produtos e quais funcionários realizaram cada venda.

3. REQUISITOS FUNCIONAIS

Diferentes grupos de usuários demandarão diferentes operações de manipulação de dados sobre diferentes partes do banco de dados.

O grupo Atendentes/Caixas demandará operações de registro e atualização de vendas, itens de venda e controle de estoque, pois são responsáveis pelo atendimento direto no ponto de venda.

O grupo Clientes terá acesso restrito ao sistema, limitado à atualização de seus dados cadastrais e consultas sobre suas compras.

O grupo Gerência demandará consultas estratégicas e relatórios consolidados, relacionados ao desempenho de vendas, lucratividade e comportamento dos clientes.

O grupo Equipe Administrativa será responsável pelo cadastro e atualização de fornecedores, produtos e funcionários, garantindo a manutenção do banco de dados em suporte às operações do supermercado.

A Tabela 1 apresenta a relação dos principais requisitos funcionais do sistema, o grupo de usuários que fará uso de cada requisito e a frequência esperada de utilização:

Tabela 1 – Requisitos Funcionais.

Consulta		Grupo	Frequência
Q001	O sistema deve permitir o cadastro de clientes.	Equipe Administrativa	M
Q002	O sistema deve permitir a atualização de dados dos clientes.	Cliente/Equipe Administrativa	B
Q003	O sistema deve permitir o cadastro de fornecedores.	Equipe Administrativa	B
Q004	O sistema deve registrar os produtos fornecidos por cada fornecedor.	Equipe Administrativa	B
Q005	O sistema deve permitir o cadastro de produtos com nome, categoria, preço e estoque.	Equipe Administrativa	M
Q006	O sistema deve permitir a atualização do estoque de produtos após uma venda.	Atendentes/Caixas	A
Q007	O sistema deve registrar as vendas realizadas, vinculando cliente e data.	Atendentes/Caixas	A
Q008	O sistema deve armazenar os itens de cada venda, com quantidade e preço unitário.	Atendentes/Caixas	A
Q009	O sistema deve calcular automaticamente o valor total da venda.	Atendentes/Caixas	A
Q010	O sistema deve permitir a consulta de produtos com estoque abaixo do mínimo.	Gerência	M
Q011	O sistema deve gerar informações das vendas por período.	Gerência	A
Q012	O sistema deve identificar os produtos mais vendidos.	Gerência	A
Q013	O sistema deve identificar os clientes que mais compraram.	Gerência	M
Q014	O sistema deve permitir a busca de clientes por nome ou CPF.	Equipe Administrativa	M
Q015	O sistema deve permitir a busca de produtos por nome ou categoria.	Atendentes/Caixas	A
Q016	O sistema deve permitir consultar o estoque agrupado por fornecedor, exibindo nome do fornecedor, categoria, quantidade em estoque e valor total investido.	Gerência	M
Q017	O sistema deve gerar informações do histórico de compras de um cliente, incluindo produtos adquiridos, quantidades, valores e período.	Equipe Administrativa	B
Q018	O sistema deve registrar o funcionário responsável por cada venda e permitir a consulta do desempenho por funcionário, incluindo quantidade de vendas, valor total e produtos mais vendidos.	Gerência	B
Q019	O sistema deve permitir a consulta de produtos que atingiram o nível mínimo de estoque, exibindo dados do produto e do fornecedor para reposição.	Gerência	A
Q020	O sistema deve calcular o valor total de vendas (faturamento) agrupado por categoria de produto.	Gerência	M

4. PROJETO CONCEITUAL

Essa seção apresenta o projeto conceitual do Sistema de Supermercado (v1.0), descrevendo as principais estruturas e restrições conceituais do banco de dados. Particularmente, a Figura 1 apresenta o diagrama entidade-relacionamento (ER) do modelo conceitual do Sistema de Supermercado (v1.0).

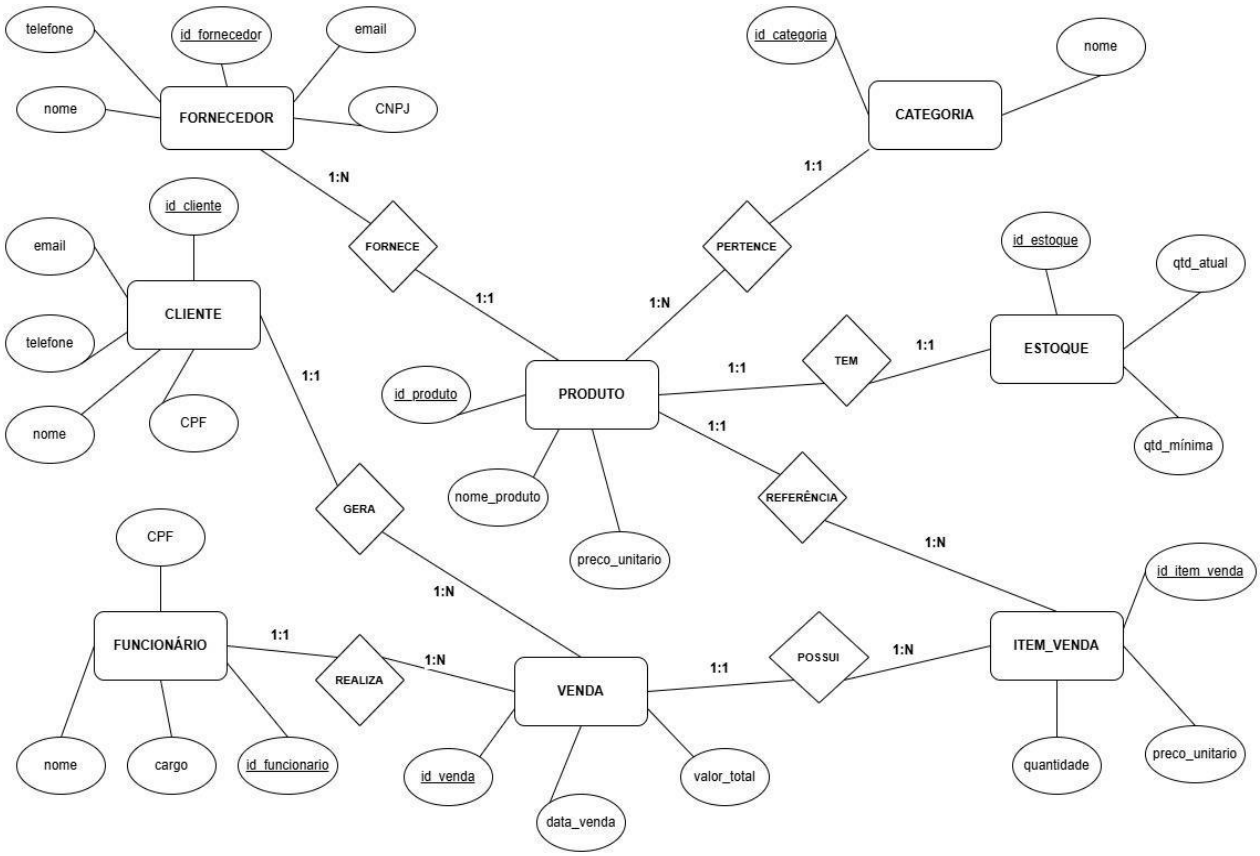

Figura 1 - Diagrama ER do modelo conceitual do banco de dados Sistema de Supermercado (v1.0) elaborado no software Draw.io.

5. PROJETO LÓGICO

Neste ponto, apresenta-se o Projeto Lógico do banco de dados Sistema de Supermercado (v1.0) que serve como o mapa detalhado para sua implementação. O Projeto Lógico onde estabeleceu-se as estruturas e restrições lógicas essenciais, definindo como o sistema será construído sob o modelo de implementação relacional. Na

Figura 2 tem-se o diagrama relacional do banco de dados baseado a partir do modelo conceitual em uma estrutura diferente focando nas chaves primárias e estrangeiras.

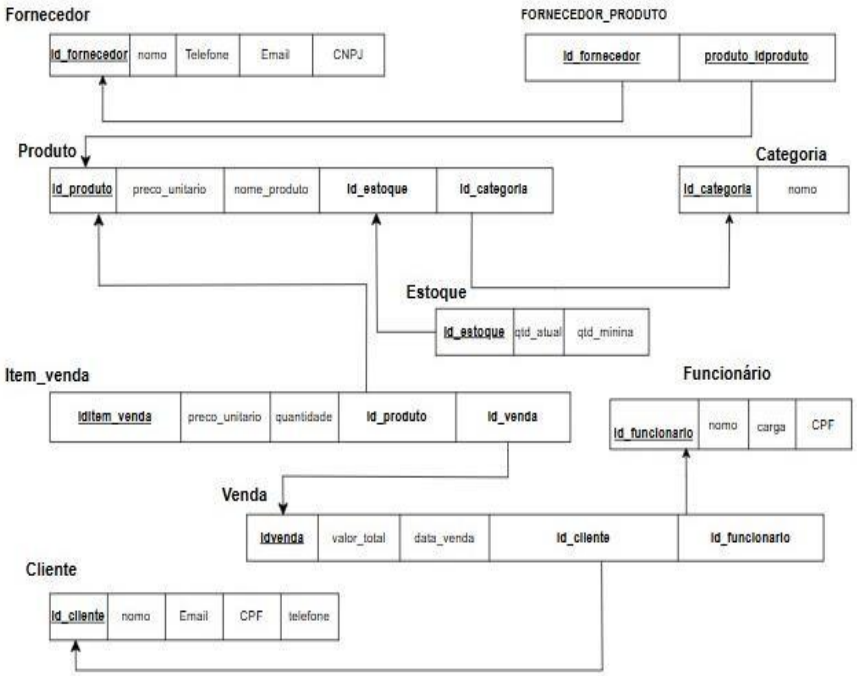

Figura 2- Diagrama ER do modelo Lógico do banco de dados Sistema de Supermercado (v1.0) elaborado no software Draw.io

O diagrama relacional ilustrado na Figura 2 não é só a estrutura estética que foi modificada, ele enfatiza novas características do banco de dados e desempenha um papel fundamental o que possibilita compreender, de maneira objetiva e compacta, as relações, atributos, restrições de chave e de integridade referencial que estruturam o banco de dados, independentemente do SGBD relacional para a sua implementação. Vale destacar que a definição de outras estruturas e restrições adicionais depende diretamente das características do SGBD comercial que poderá ser adotado.

No processo de modelagem lógica, além da construção do modelo de dados, foi definido qual abordagem e solução tecnológica serão utilizadas. Para o Sistema de Supermercado (v1.0), a opção definida foi a implementação em um SGBD comercial, o MySQL.

A próxima imagem, a Figura 3, apresenta o Modelo de diagrama Entidade-Relacionamento Estendido (EER) correspondente ao modelo do Sistema de

Supermercado (v1.0), no qual estão representadas as restrições aplicadas ao banco de dados por meio de pequenos símbolos visuais “nas linhas das caixas azuis” que facilitam sua interpretação: a chave primária é exibida como um ícone de chave amarela à esquerda do atributo; o tipo de dado é mostrado à direita do rótulo; a nulidade é indicada por losangos (branco para atributos que permitem valores nulos e azul para atributos obrigatórios); e a integridade referencial é representada por losangos vermelhos que sinalizam as chaves estrangeiras.

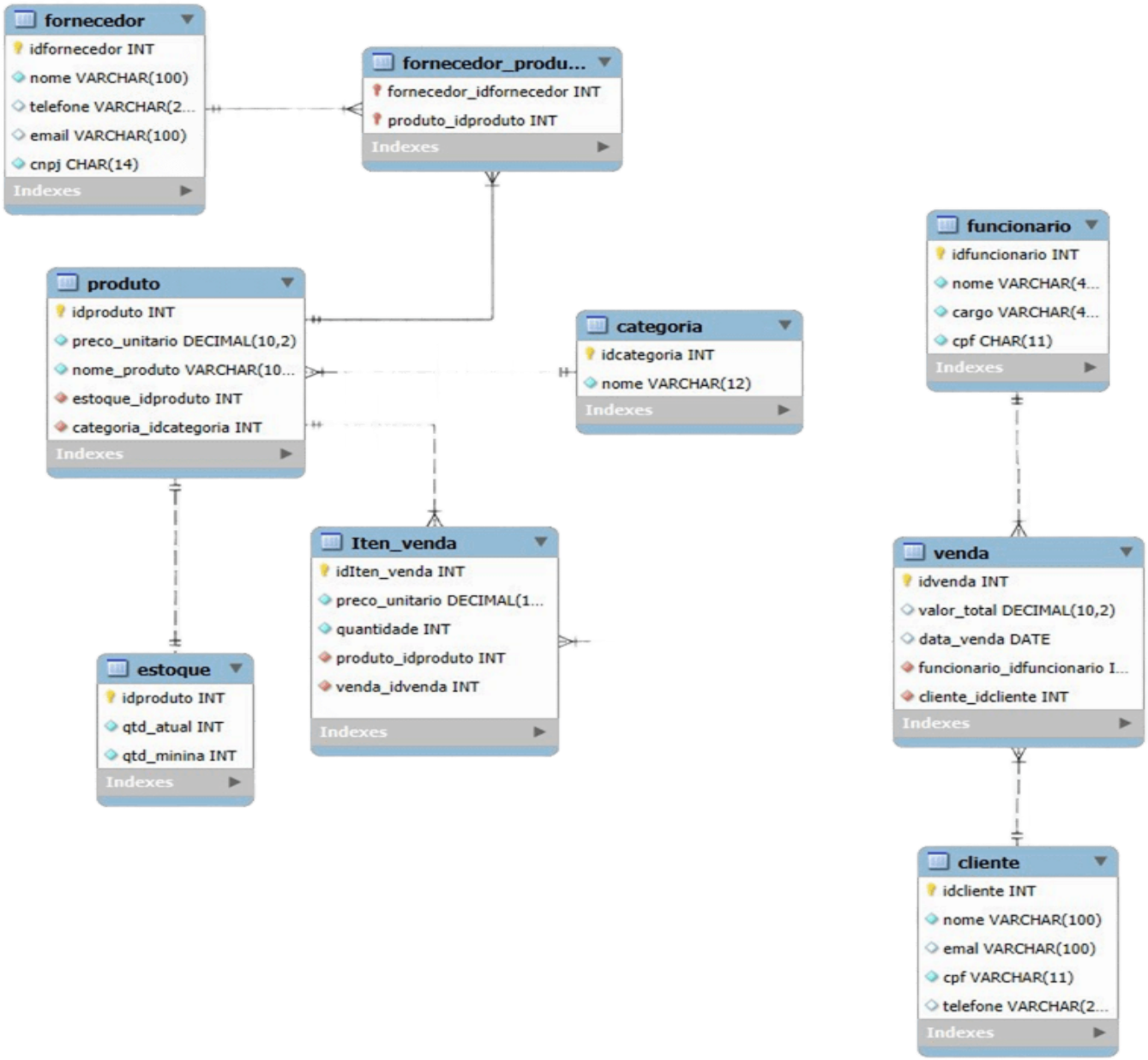

Figura 3 - Diagrama EER do banco de dados Sistema de Supermercado (v1.0) elaborado no MYSQL.

6. SOFTARES USADOS

O diagrama Entidade Relacionamento (ER) do modelo conceitual do Sistema de Supermercado (v1.0) foi construído no software Draw.io, software escolhido pela praticidade e pela possibilidade de organizar visualmente os principais elementos do banco de dados ainda em sua fase inicial. O uso da ferramenta permitiu representar, de forma clara e acessível, as entidades, seus atributos e os relacionamentos fundamentais. Além disso, o ambiente gráfico do Draw.io facilitou ajustes durante a análise, como a inclusão ou retirada de entidades ou a revisão de cardinalidades, tornando o processo de modelagem mais dinâmico e flexível. Essa etapa forneceu uma visão abrangente do domínio do problema e serviu como base sólida para as etapas seguintes da modelagem.

A partir do modelo conceitual, o modelo lógico também foi desenvolvido no Draw.io, aproveitando a familiaridade adquirida na fase anterior e garantindo continuidade ao processo de construção do banco de dados. Nessa etapa, o diagrama passou a assumir uma estrutura mais detalhada. O Draw.io contribuiu para a parte de visualizar melhor das chaves primárias e estrangeiras.

Já o diagrama estendido (EER), foi elaborado no MySQL Workbench, ferramenta que possibilitou os recursos para modelagem. O Workbench permitiu refinar o modelo, adicionando elementos como tipos de dados. A ferramenta também forneceu uma visualização mais precisa da estrutura que seria implantada no MySQL, o que facilitou a transição para a etapa física do banco de dados. Com isso, o EER desenvolvido garante que o modelo estivesse não apenas conceitualmente correto, mas também preparado para atender às exigências práticas das futuras aplicações.

Por fim, para a transformação do projeto em notebook do Python, foi utilizado o O Google Colab juntamente com a biblioteca Pandas. A escolha do Colab visou o desenvolvimento do trabalho em equipe, visto que a ferramenta possibilita edição colaborativa.

Durante o desenvolvimento do notebook do Python, foram encontradas algumas dificuldades na formatação dos arquivos CSV e nos nomes de algumas colunas, que não seguiam uma padronização. Ao identificar essas pontualidades, foi necessário corrigir,

utilizado o recurso de renomeação de colunas da biblioteca Pandas e os DataFrames passaram por uma reorganização antes da execução das consultas.

Concluídas as correções, iniciou-se o processo de validação. Cada consulta desenvolvida no notebook foi testada individualmente, permitindo a comparação direta entre os resultados obtidos no ambiente Python e as que estavam previamente registradas no MySQL. Essa verificação garantia a consistência das informações e reforçava a confiabilidade do procedimento adotado.

7. SCRIPTS, CONSULTAS, VIEWS E RESULTADOS

O script de criação do banco de dados do Sistema de Supermercado (v1.0) foi desenvolvido no MySQL Workbench/Dbeaver, constituindo uma etapa essencial para a materialização da estrutura previamente planejada. Nesse momento, tabelas, chaves primárias, relacionamentos e restrições passaram do plano conceitual para a implementação efetiva.

Da mesma forma, o script de inserção de dados foi construído nas mesmas ferramentas, com o objetivo de povoar o banco e verificar o comportamento do sistema em condições próximas às práticas de um supermercado. A inclusão dos registros exigiu atenção à coerência entre os tipos de dados, ao alinhamento entre tabelas relacionadas e à integridade referencial.

Por fim, foram desenvolvidas as views responsáveis por apresentar os resultados das consultas e dos requisitos funcionais implementados. As views atuaram como uma camada de apresentação, permitindo o acesso a informações relevantes sem a necessidade de manipular diretamente as tabelas base. Essa abordagem evidenciou que a modelagem, os scripts e as regras definidas ao longo do projeto estavam operando adequadamente. Assim, essa etapa reforça a importância de um planejamento detalhado nas fases iniciais do desenvolvimento. Abaixo segue os resultados.

1. Criação da View do Requisito Q001 - O sistema deve permitir o cadastro de clientes.

Query 1

```
-- Criação da View do Requisito Q001 - O sistema deve permitir o cadastro de clientes.
DROP VIEW IF EXISTS vw_q001_clientes;
CREATE VIEW vw_q001_clientes AS
SELECT
  c.idcliente,
  c.nome,
  c.email,
  c.cpf,
  c.telefone
FROM cliente c;
SELECT * FROM Sistema.vw_q001_clientes;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idcliente	nome	email	cpf	telefone
1	Maria de Fátima Silva	maria.silva@email.com	12345678901	(11) 98765-1234
2	João Pedro Santos	joao.santos@emai	maria.silva@email.com	(21) 91234-5678
3	Ana Paula Costa	ana.paula@email.com	34567890123	(31) 99876-5432
4	Carlos Alberto Souza	carlos.souza@email.com	45678901234	(41) 91122-3344
5	Beatriz Oliveira Lima	beatriz.lima@email.com	56789012345	(51) 95566-7788
6	Ricardo José Almeida	ricardo.almeida@email.com	67890123456	(61) 93344-5566
7	Fernanda Gomes Pereira	fernanda.gomes@email.com	78901234567	(71) 97788-9900
8	Guilherme Rocha Ferreira	guilherme.rocha@email.com	89012345678	(81) 92233-4455
9	Helena Martins Dias	helena.martins@email.com	90123456789	(91) 96677-8899
10	Luís Felipe Ribeiro	luis.ribeiro@email.com	01234567890	(11) 91011-2131
11	Patrícia Nunes Barbosa	patricia.barbosa@email.com	10987654321	(19) 91213-1415
12	Roberto Cavalcanti Teles	roberto.teles@email.com	21098765432	(21) 91516-1718
13	Silvana Xavier Duarte	silvana.duarte@email.com	32109876543	(31) 91819-2021
14	Thiago Ventura Mendes	thiago.mendes@email.com	43210987654	(41) 92122-2324
15	Vanessa Cruz Freitas	vanessa.freitas@email.com	54321098765	(51) 92425-2627
16	Andréia Correia Pires	andreia.pires@email.com	65432109876	(61) 92728-2930
17	Bruno Lopes Morais	bruno.morais@email.com	76543210987	(71) 93031-3233
18	Cintia Regina Santos	cintia.santos@email.com	87654321098	(81) 93334-3536
19	Daniel Rocha Barbosa	daniel.barbosa@email.com	98765432109	(91) 93637-3839
20	Erica Pires Mendes	erica.mendes@email.com	09876543210	(11) 93940-4142
21	Fábio Guedes Pereira	fabio.pereira@email.com	10293847561	(19) 94243-4445
22	Gisele Lima Oliveira	gisele.oliveira@email.com	21304958672	(21) 94546-4748
23	Hugo Mendes Silva	hugo.silva@email.com	32415069783	(31) 94849-5051

vw_q001_clientes 1

Figura 4 - Cadastro de Clientes elaborado no software MySQL Workbench.

2. Criação da View do Requisito Q002 - O sistema deve permitir a atualização de dados dos clientes. (Não houve necessidade de atualização).

3. Criação da View do Requisito Q003 - O sistema deve permitir o cadastro de fornecedores.

SQL File 1* | vw_q003_fornecedores

```sql
-- Criação da View do Requisito Q003 - O sistema deve permitir o cadastro de fornecedores.
DROP VIEW IF EXISTS vw_q003_fornecedores;
CREATE VIEW vw_q003_fornecedores AS
SELECT
  f.idfornecedor,
  f.nome,
  f.telefone,
  f.email,
  f.cnpj
FROM fornecedor f;
SELECT * FROM Sistema.vw_q003_fornecedores;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idfornecedor	nome	telefone	email	cnpj
1	Distribuidora Alimentos BR	(11) 98765-4321	contato@alimentosbr.com	01.234.567/0001-89
2	Laticínios Do Campo S.A.	(19) 91234-5678	vendas@docampo.com	02.345.678/0001-00
3	Frigorífico Boi Forte	(21) 99876-5432	comercial@boiforte.com	03.456.789/0001-11
4	Grãos Essenciais Ltda	(31) 91122-3344	pedidos@graosessenciais.com	04.567.890/0001-22
5	Engarrafadora Aurora	(41) 95566-7788	distribuidora@aurora.com	05.678.901/0001-33
6	Mundo Gelado Distribuição	(51) 93344-5566	frios@mundogelado.com	06.789.012/0001-44
7	Cosméticos e Cia	(61) 97788-9900	higiene@cosmeticosecia.com	07.890.123/0001-55
8	Química Limpeza Total	(71) 92233-4455	limpeza@quimicatotal.com	08.901.234/0001-66
9	Panificadora Gourmet Fornecimento	(81) 96677-8899	gourmet@panificadora.com	09.012.345/0001-77
10	Doces Delícia Atacado	(91) 91011-2131	atacado@docesdelicia.com	10.123.456/0001-88
11	Alimentos Frescos SC	(48) 91213-1415	fresh@alimentossc.com	11.234.567/0001-99
12	Queijos e Vinhos Sul	(54) 91516-1718	queijosevinhos@sul.com	12.345.678/0001-01
13	Açougue Central	(62) 91819-2021	contato@acouguecentral.com	13.456.789/0001-12
14	Cereais Matinais BR	(85) 92122-2324	vendas@cereaisbr.com	14.567.890/0001-23
15	Sucos Naturais da Serra	(27) 92425-2627	pedidos@sucosnaturais.com	15.678.901/0001-34
16	Polpas e Sorvetes RJ	(24) 92728-2930	comercial@polpasrj.com	16.789.012/0001-45
17	Sabonetes Finos SP	(12) 93031-3233	higiene@sabonetesfinos.com	17.890.123/0001-56
18	Desinfetantes Potência	(13) 93334-3536	vendas@desinfetantes.com	18.901.234/0001-67
19	Massas e Pães Gourmet	(14) 93637-3839	fornecedor@massasepaes.com	19.012.345/0001-78
20	Chocolateira Pura Tentação	(15) 93940-4142	contato@puratentacao.com	20.123.456/0001-80
21	Legumes Frescos MG	(32) 94243-4445	pedidos@legumesmg.com	21.234.567/0001-91
22	Derivados de Leite Sul	(47) 94546-4748	vendas@derivadosdeleite.com	22.345.678/0001-02
23	Peixes e Frutos do Mar	(84) 94849-5051	frutosdomar@peixes.com	23.456.789/0001-13
24	Biscoitos e Bolachas SA	(82) 95152-5354	biscoitos@bolachas.com	24.567.890/0001-24

vw_q003_fornecedores 2

Figura 5 - Cadastro de Fornecedores no software MySQL Workbench.

4. Criação da View do Requisito Q004 - O sistema deve registrar os produtos fornecidos por cada fornecedor. (N:N).

SQL File 1*

```sql
-- Criação da View do Requisito Q004 - O sistema deve registrar os produtos fornecidos por cada fornecedor. (N:N)
DROP VIEW IF EXISTS vw_q004_produtos_por_fornecedor;
CREATE VIEW vw_q004_produtos_por_fornecedor AS
SELECT
  f.idfornecedor,
  f.nome AS fornecedor,
  p.idproduto,
  p.nome_produto,
  p.preco_unitario,
  cat.idcategoria,
  cat.nome AS categoria
FROM fornecedor f
JOIN fornecedor_produto fp ON fp.fornecedor_idfornecedor = f.idfornecedor
JOIN produto p ON p.idproduto = fp.produto_idproduto
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria;
-- Exemplo:
SELECT * FROM vw_q004_produtos_por_fornecedor WHERE idfornecedor
```

Result Grid

idfornecedor	fornecedor	idproduto	nome_produto	preco_unitario	idcategoria	categoria
1	Distribuidora Alimentos BR	1	Banana Nanica (Kg)	5.99	1	Hortifrúti
11	Alimentos Frescos SC	1	Banana Nanica (Kg)	5.99	1	Hortifrúti
21	Legumes Frescos MG	2	Alface Crespa (Unidade)	3.50	1	Hortifrúti
11	Alimentos Frescos SC	3	Batata Inglesa (Kg)	7.89	1	Hortifrúti
41	Frutas Exóticas Norte	4	Tomate Salada (Kg)	6.20	1	Hortifrúti
1	Distribuidora Alimentos BR	5	Maçã Fuji (Kg)	9.99	1	Hortifrúti
2	Laticínios Do Campo S.A.	6	Leite Integral 1L	4.59	2	Laticínios e Frios
12	Queijos e Vinhos Sul	7	Queijo Muçarela (Kg)	38.90	2	Laticínios e Frios
32	Iogurtes e Sobremesas	8	Iogurte Natural 170g	2.99	2	Laticínios e Frios
42	Manteigas e Margarinas	9	Manteiga com Sal 200g	12.50	2	Laticínios e Frios
2	Laticínios Do Campo S.A.	10	Presunto Cozido (Kg)	25.90	2	Laticínios e Frios
12	Queijos e Vinhos Sul	10	Presunto Cozido (Kg)	25.90	2	Laticínios e Frios
3	Frigorífico Boi Forte	11	Acém Bovino (Kg)	29.90	3	Carnes e Aves
33	Aves Premium	12	Peito de Frango (Kg)	18.90	3	Carnes e Aves
43	Embutidos Nobres	13	Linguiça Calabresa Def...	22.50	3	Carnes e Aves
23	Peixes e Frutos do Mar	14	Filé de Tilápia Congelad...	35.90	3	Carnes e Aves

vw_q004_produtos_por_fomec...

Figura 6 - Registro dos produtos fornecidos por cada fornecedor.

5. Criação da View do Requisito Q005 - O sistema deve permitir o cadastro de produtos com nome, categoria, preço e estoque.

SQL File 1*

```sql
-- Criação da View do Requisito Q005 - O sistema deve permitir o cadastro de produtos com nome, categoria, preço e estoque.
DROP VIEW IF EXISTS vw_q005_produtos_detalhado;
CREATE VIEW vw_q005_produtos_detalhado AS
SELECT
  p.idproduto,
  p.nome_produto,
  p.preco_unitario,
  cat.idcategoria,
  cat.nome AS categoria,
  e.qtd_atual AS qtd_estoque,
  e.qtd_minima AS qtd_minima
FROM produto p
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria
LEFT JOIN estoque e ON e.idproduto = p.estoque_idproduto;
-- Exemplo:
SELECT * FROM vw_q005_produtos_detalhado;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idproduto	nome_produto	preco_unitario	idcategoria	categoria	qtd_estoque	qtd_minima
1	Banana Nanica (Kg)	5.99	1	Hortifrúti	150	30
2	Alface Crespa (Unidade)	3.50	1	Hortifrúti	200	50
3	Batata Inglesa (Kg)	7.89	1	Hortifrúti	180	40
4	Tomate Salada (Kg)	6.20	1	Hortifrúti	160	35
5	Maçã Fuji (Kg)	9.99	1	Hortifrúti	120	25
6	Leite Integral 1L	4.59	2	Laticínios e Frios	400	80
7	Queijo Muçarela (Kg)	38.90	2	Laticínios e Frios	500	100
8	Iogurte Natural 170g	2.99	2	Laticínios e Frios	350	70
9	Manteiga com Sal 200g	12.50	2	Laticínios e Frios	250	50
10	Presunto Cozido (Kg)	25.90	2	Laticínios e Frios	300	60
11	Acém Bovino (Kg)	29.90	3	Carnes e Aves	40	50
12	Peito de Frango (Kg)	18.90	3	Carnes e Aves	320	60
13	Linguiça Calabresa Def...	22.50	3	Carnes e Aves	280	55
14	Filé de Tilápia Congelad...	35.90	3	Carnes e Aves	1	30
15	Ovo Branco Grande (Dú...	10.90	3	Carnes e Aves	300	60
16	Arroz Agulhinha 5kg	24.99	4	Mercearia Básica	500	100
17	Feijão Carioca 1kg	8.50	4	Mercearia Básica	600	120

vw_q005_produtos_detalhado 6

Figura 7 - Cadastro de produtos com nome, categoria, preço e estoque.

6. Criação da View do Requisito Q006 - O sistema deve permitir a atualização do estoque de produtos após uma venda.

SQL File 1*

```sql
-- Criação da View do Requisito Q006 - O sistema deve permitir a atualização do estoque de produtos após uma venda.
DROP VIEW IF EXISTS vw_q006_impacto_vendas_no_estoque;
CREATE VIEW vw_q006_impacto_vendas_no_estoque AS
SELECT
  p.idproduto,
  p.nome_produto,
  e.qtd_atual AS qtd_estoque_atual,
  SUM(iv.quantidade) AS qtd_vendida_total,
  COALESCE(e.qtd_atual - SUM(iv.quantidade), e.qtd_atual) AS qtd_estoque_pos_venda
FROM produto p
LEFT JOIN estoque e ON e.idproduto = p.estoque_idproduto
LEFT JOIN item_venda iv ON iv.produto_idproduto = p.idproduto
GROUP BY p.idproduto, p.nome_produto, e.qtd_atual;
-- Exemplo:
SELECT * FROM vw_q006_impacto_vendas_no_estoque;
```

Result Grid

idproduto	nome_produto	qtd_estoque_atual	qtd_vendida_total	qtd_estoque_pos_venda
1	Banana Nanica (Kg)	150	2	148
2	Alface Crespa (Unidade)	200	5	195
3	Batata Inglesa (Kg)	180	1	179
4	Tomate Salada (Kg)	160	6	154
5	Maçã Fuji (Kg)	120	7	113
6	Leite Integral 1L	400	9	391
7	Queijo Muçarela (Kg)	500	1	499
8	Iogurte Natural 170g	350	8	342
9	Manteiga com Sal 200g	250	1	249
10	Presunto Cozido (Kg)	300	7	293
11	Acém Bovino (Kg)	40	8	32
12	Peito de Frango (Kg)	320	4	316
13	Linguiça Calabresa Def...	280	3	277
14	Filé de Tilápia Congelad...	1	2	-1
15	Ovo Branco Grande (Dú...	300	5	295
16	Arroz Agulhinha 5kg	500	8	492
17	Feijão Carioca 1kg	600	3	597
18	Açúcar Refinado 1kg	450	11	439
19	Óleo de Soia 900ml	400	5	395

vw_q006_impacto_vendas_no_...

Figura 8 - Atualização do estoque de produtos após uma venda.

7. Criação da View do Requisito Q007 - O sistema deve registrar as vendas realizadas, vinculando cliente e data (vendas cabeçalho com cliente e funcionário).

SQL File 1*

```sql
-- Criação da View do Requisito Q007 - O sistema deve registrar as vendas realizadas, vinculando cliente e data.
-- (vendas cabeçalho com cliente e funcionário.)
DROP VIEW IF EXISTS vw_q007_vendas_cabecalho;
CREATE VIEW vw_q007_vendas_cabecalho AS
SELECT
  v.idvenda,
  v.data_venda,
  v.valor_total,
  v.cliente_idcliente,
  c.nome AS nome_cliente,
  v.funcionario_idfuncionario,
  f.nome AS nome_funcionario
FROM venda v
LEFT JOIN cliente c ON c.idcliente = v.cliente_idcliente
LEFT JOIN funcionario f ON f.idfuncionario = v.funcionario_idfuncionario;
-- Exemplo:
SELECT * FROM vw_q007_vendas_cabecalho;
```

Result Grid

idvenda	data_venda	valor_total	cliente_idcliente	nome_cliente	funcionario_idfuncionario	nome_funcionario
1	2025-01-05	45.98	1	Maria de Fátima Silva	3	Marcelo Aguiar Neto
2	2025-01-05	120.50	5	Beatriz Oliveira Lima	4	Jéssica Soares Reis
3	2025-01-06	78.25	10	Luís Felipe Ribeiro	3	Marcelo Aguiar Neto
4	2025-01-07	22.90	2	João Pedro Santos	4	Jéssica Soares Reis
5	2025-01-08	55.70	8	Guilherme Rocha Ferreira	3	Marcelo Aguiar Neto
6	2025-01-09	15.00	15	Vanessa Cruz Freitas	4	Jéssica Soares Reis
7	2025-01-10	98.30	20	Erica Pires Mendes	3	Marcelo Aguiar Neto
8	2025-01-11	33.45	3	Ana Paula Costa	4	Jéssica Soares Reis
9	2025-01-12	110.00	12	Roberto Cavalcanti Teles	3	Marcelo Aguiar Neto
10	2025-01-15	65.50	4	Carlos Alberto Souza	4	Jéssica Soares Reis
11	2025-02-01	88.90	25	Júlio César Freitas	3	Marcelo Aguiar Neto
12	2025-02-02	40.25	30	Otávio Morais Duarte	4	Jéssica Soares Reis
13	2025-02-03	15.99	1	Maria de Fátima Silva	3	Marcelo Aguiar Neto
14	2025-02-04	190.10	18	Cintia Regina Santos	4	Jéssica Soares Reis
15	2025-02-05	50.00	7	Fernanda Gomes Pereira	3	Marcelo Aguiar Neto
16	2025-02-08	72.80	11	Patrícia Nunes Barbosa	4	Jéssica Soares Reis
17	2025-02-10	25.60	9	Helena Martins Dias	3	Marcelo Aguiar Neto

vw_q007_vendas_cabecalho 10

Figura 9 - registrar as vendas realizadas, vinculando cliente e data (vendas cabeçalho com cliente e funcionário).

8. Criação da View do Requisito Q008 - O sistema deve armazenar os itens de cada venda, com quantidade e preço unitário.

SQL File 1*

```sql
-- Criação da View do Requisito Q008 - O sistema deve armazenar os itens de cada venda, com quantidade e preço unitário.
DROP VIEW IF EXISTS vw_q008_itens_por_venda;
CREATE VIEW vw_q008_itens_por_venda AS
SELECT
  iv.iditem_venda,
  iv.venda_idvenda,
  iv.produto_idproduto,
  p.nome_produto,
  iv.quantidade,
  iv.preco_unitario,
  (iv.quantidade * iv.preco_unitario) AS subtotal_item
FROM item_venda iv
JOIN produto p ON p.idproduto = iv.produto_idproduto;
-- Exemplo:
SELECT * FROM vw_q008_itens_por_venda WHERE venda_idvenda = 23;
```

Result Grid

iditem_venda	venda_idvenda	produto_idproduto	nome_produto	quantidade	preco_unitario	subtotal_item
47	23	6	Leite Integral 1L	4	4.59	18.36
48	23	16	Arroz Agulhinha 5kg	2	24.99	49.98
49	23	11	Acém Bovino (Kg)	3	29.90	89.70
50	23	30	Lasanha Bolonhesa Congelada 600g	1	21.50	21.50

Figura 10 - Armazenamento dos itens de cada venda, com quantidade e preço unitário.

9. Criação da View do Requisito Q009 - O sistema deve calcular automaticamente o valor total da venda. Recalcula o total por venda a partir dos itens (valida valor_total).

SQL File 1*

```sql
-- Criação da View do Requisito Q009 - O sistema deve calcular automaticamente o valor total da venda.
-- Recalcula o total por venda a partir dos itens (valida valor_total).
DROP VIEW IF EXISTS vw_q009_calculo_valor_total_venda;
CREATE VIEW vw_q009_calculo_valor_total_venda AS
SELECT
  v.idvenda,
  v.valor_total AS valor_total_registrado,
  COALESCE(SUM(iv.quantidade * iv.preco_unitario), 0) AS valor_total_calculado,
  (v.valor_total - COALESCE(SUM(iv.quantidade * iv.preco_unitario), 0)) AS diferenca
FROM venda v
LEFT JOIN item_venda iv ON iv.venda_idvenda = v.idvenda
GROUP BY v.idvenda, v.valor_total;
-- Exemplo:
SELECT * FROM vw_q009_calculo_valor_total_venda WHERE diferenca <> 0;
```

Result Grid

idvenda	valor_total_registrado	valor_total_calculado	diferenca
1	45.98	40.16	5.82
2	120.50	94.60	25.90
3	78.25	66.99	11.26
4	22.90	13.00	9.90
5	55.70	55.28	0.42
6	15.00	14.00	1.00
7	98.30	93.70	4.60
8	33.45	33.40	0.05
9	110.00	103.20	6.80
10	65.50	50.60	14.90
11	88.90	66.20	22.70
12	40.25	34.95	5.30
13	15.99	7.89	8.10
14	190.10	100.40	89.70
15	50.00	19.98	30.02
16	72.80	36.65	36.15
17	25.60	23.92	1.68
18	130.40	109.60	20.80
19	10.99	5.00	5.99
20	95.70	59.50	36.20

vw_q009_calculo_valor_total_v...

Figura 11 - Cálculo automático do valor total da venda.

10. Criação da View do Requisito Q010 - O sistema deve permitir a consulta de produtos com estoque abaixo do mínimo. Alerta de estoque baixo (qtd_atual <= qtd_minima).

SQL File 1*

```sql
-- Criação da View do Requisito Q010 - O sistema deve permitir a consulta de produtos com estoque abaixo do mínimo.
-- alerta de estoque baixo (qtd_atual <= qtd_minima).
DROP VIEW IF EXISTS vw_q010_produtos_estoque_baixo;
CREATE VIEW vw_q010_produtos_estoque_baixo AS
SELECT
  p.idproduto,
  p.nome_produto,
  cat.nome AS categoria,
  e.qtd_atual,
  e.qtd_minima,
  (e.qtd_minima - e.qtd_atual) AS qtd_para_repor
FROM produto p
JOIN estoque e ON e.idproduto = p.estoque_idproduto
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria
WHERE e.qtd_atual <= e.qtd_minima;
-- Exemplo:
SELECT * FROM vw_q010_produtos_estoque_baixo ORDER BY qtd_para_repor DESC;
```

Result Grid

idproduto	nome_produto	categoria	qtd_atual	qtd_minima	qtd_para_repor
14	Filé de Tilápia Congelado 500g	Carnes e Aves	1	30	29
11	Acém Bovino (Kg)	Carnes e Aves	40	50	10
27	Pizza Muçarela Congelada	Congelados	23	25	2

Figura 12 - Alerta de estoque baixo.

11. Criação da View do Requisito Q011 - O sistema deve gerar informações das vendas por período. Agregada por dia (data_venda): número de vendas e faturamento).

SQL File 1*

```sql
-- Criação da View do Requisito Q011 - O sistema deve gerar informações das vendas por período.
-- agregada por dia (data_venda): número de vendas e faturamento.
DROP VIEW IF EXISTS vw_q011_vendas_por_dia;
CREATE VIEW vw_q011_vendas_por_dia AS
SELECT
  v.data_venda,
  COUNT(v.idvenda) AS numero_vendas,
  SUM(COALESCE(v.valor_total,0)) AS faturamento_total,
  AVG(COALESCE(v.valor_total,0)) AS ticket_medio
FROM venda v
GROUP BY v.data_venda
ORDER BY v.data_venda;
-- Exemplo:
SELECT * FROM vw_q011_vendas_por_dia WHERE data_venda BETWEEN '2025-02-01' AND '2025-02-28';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

data_venda	numero_vendas	faturamento_total	ticket_medio
2025-02-01	1	88.90	88.900000
2025-02-02	1	40.25	40.250000
2025-02-03	1	15.99	15.990000
2025-02-04	1	190.10	190.100000
2025-02-05	1	50.00	50.000000
2025-02-08	1	72.80	72.800000
2025-02-10	1	25.60	25.600000
2025-02-14	1	130.40	130.400000
2025-02-20	1	10.99	10.990000
2025-02-28	1	95.70	95.700000

Figura 13 - Gerar informações das vendas por período. Agregada por dia (data_venda): número de vendas e faturamento).

12. Criação da View do Requisito Q012 - O sistema deve identificar os produtos mais vendidos. Ranking por quantidade e receita total (todos os tempos).

SQL File 1*

```sql
-- Criação da View do Requisito Q012 - O sistema deve identificar os produtos mais vendidos.
-- Ranking por quantidade e receita total (todos os tempos).
DROP VIEW IF EXISTS vw_q012_produtos_mais_vendidos;
CREATE VIEW vw_q012_produtos_mais_vendidos AS
SELECT
  p.idproduto,
  p.nome_produto,
  cat.nome AS categoria,
  SUM(iv.quantidade) AS quantidade_total_vendida,
  SUM(iv.quantidade * iv.preco_unitario) AS receita_total
FROM item_venda iv
JOIN produto p ON p.idproduto = iv.produto_idproduto
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria
GROUP BY p.idproduto, p.nome_produto, cat.nome
ORDER BY quantidade_total_vendida DESC, receita_total DESC;
-- Exemplo:
SELECT * FROM vw_q012_produtos_mais_vendidos LIMIT 15;
```

Result Grid

idproduto	nome_produto	categoria	quantidade_total_vendida	receita_total
24	Água Mineral sem Gás 1,5L	Bebidas	30	75.00
22	Cerveja Pilsen Lata 350ml	Bebidas	20	69.80
36	Detergente Líquido 500ml	Limpeza Doméstica	16	44.80
37	Água Sanitária 1L	Limpeza Doméstica	11	53.90
18	Açúcar Refinado 1kg	Mercearia Básica	11	46.20
20	Macarrão Espaguete 500g	Mercearia Básica	11	41.25
47	Pacote de Balas Sortidas 500g	Doces e Guloseimas	10	119.00
41	Pão Francês (Unidade)	Padaria	10	8.00
49	Chiclete Bola Tutti Frutti	Doces e Guloseimas	10	5.00
28	Batata Palito Congelada 400g	Congelados	9	116.91
6	Leite Integral 1L	Laticínios e Frios	9	41.31
11	Acém Bovino (Kg)	Carnes e Aves	8	239.20
16	Arroz Agulhinha 5kg	Mercearia Básica	8	199.92
34	Papel Higiênico Folha Dupla 1...	Higiene Pessoal	8	183.20
48	Biscoito Recheado Chocolate	Doces e Guloseimas	8	25.60

vw_q012_produtos_mais_vendi...

Figura 14 - Identificar os produtos mais vendidos. Ranking por quantidade e receita total (todos os tempos).

13. Criação da View do Requisito Q013 - O sistema deve identificar os clientes que mais compraram. Ranking por gasto total e número de compras.

SQL File 1*

```sql
-- Criação da View do Requisito Q013 - O sistema deve identificar os clientes que mais compraram.
-- Ranking por gasto_total e número de compras.
DROP VIEW IF EXISTS vw_q013_clientes_que_mais_compraram;
CREATE VIEW vw_q013_clientes_que_mais_compraram AS
SELECT
  c.idcliente,
  c.nome,
  COUNT(v.idvenda) AS numero_de_compras,
  SUM(COALESCE(v.valor_total,0)) AS gasto_total
FROM cliente c
LEFT JOIN venda v ON v.cliente_idcliente = c.idcliente
GROUP BY c.idcliente, c.nome
ORDER BY gasto_total DESC, numero_de_compras DESC;
-- Exemplo:
SELECT * FROM vw_q013_clientes_que_mais_compraram LIMIT 15;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idcliente	nome	numero_de_compras	gasto_total
18	Cintia Regina Santos	2	370.10
10	Luís Felipe Ribeiro	2	183.40
50	Ingrid Mendes Silva	2	176.30
20	Erica Pires Mendes	2	172.80
16	Andréia Correia Pires	1	160.30
5	Beatriz Oliveira Lima	2	148.40
21	Fábio Guedes Pereira	1	145.20
30	Otávio Morais Duarte	2	140.15
45	Diego Santos Lima	2	135.40
40	Yara Pereira Mendes	2	131.39
22	Gisele Lima Oliveira	1	130.40
8	Guilherme Rocha Fer...	2	126.00
12	Roberto Cavalcanti ...	2	120.99
1	Maria de Fátima Silva	3	115.37
25	Júlio César Freitas	2	114.40

Figura 15 - Clientes que mais compraram. Ranking por gasto total e número de compras.

14. Criação da View do Requisito Q014 - O sistema deve permitir a busca de clientes por nome ou CPF.

SQL File 1*

```sql
-- Criação da View do Requisito Q014 - O sistema deve permitir a busca de clientes por nome ou CPF.
DROP VIEW IF EXISTS vw_q014_busca_clientes;
CREATE VIEW vw_q014_busca_clientes AS
SELECT
  c.idcliente,
  c.nome,
  c.cpf,
  c.email,
  c.telefone,
  COALESCE(MAX(v.data_venda), NULL) AS data_ultima_compra
FROM cliente c
LEFT JOIN venda v ON v.cliente_idcliente = c.idcliente
GROUP BY c.idcliente, c.nome, c.cpf, c.email, c.telefone;
-- Exemplo:
SELECT * FROM vw_q014_busca_clientes WHERE nome LIKE '%João%' OR cpf = '23456789012';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

idcliente	nome	cpf	email	telefone	data_ultima_compra
2	João Pedro Santos	23456789012	joao.santos@email.com	(21) 91234-5678	2025-08-10

Figura 16 - Busca de clientes por nome ou CPF.

15. Criação da View do Requisito Q015 - O sistema deve permitir a busca de produtos por nome ou categoria. Busca e filtro (usuário aplica WHERE).

SQL File 1*

```sql
-- Criação da View do Requisito Q015 - O sistema deve permitir a busca de produtos por nome ou categoria.
-- Busca e filtro (usuário aplica WHERE).
DROP VIEW IF EXISTS vw_q015_busca_produtos;
CREATE VIEW vw_q015_busca_produtos AS
SELECT
  p.idproduto,
  p.nome_produto,
  p.preco_unitario,
  cat.idcategoria,
  cat.nome AS categoria,
  e.qtd_atual AS qtd_estoque
FROM produto p
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria
LEFT JOIN estoque e ON e.idproduto = p.estoque_idproduto;
-- Exemplo:
SELECT * FROM vw_q015_busca_produtos WHERE nome_produto LIKE '%Arroz%' OR categoria = 'Mercearia Básica';
```

Result Grid

idproduto	nome_produto	preco_unitario	idcategoria	categoria	qtd_estoque
16	Arroz Agulhinha 5kg	24.99	4	Mercearia Básica	500
17	Feijão Carioca 1kg	8.50	4	Mercearia Básica	600
18	Açúcar Refinado 1kg	4.20	4	Mercearia Básica	450
19	Óleo de Soja 900ml	6.99	4	Mercearia Básica	400
20	Macarrão Espaguete 500g	3.75	4	Mercearia Básica	550

Figura 17 - Busca de produtos por nome ou categoria. Busca e filtro (usuário aplica WHERE).

16. Criação da View do Requisito Q016 - O sistema deve permitir consultar o estoque agrupado por fornecedor exibindo nome do fornecedor, categoria, quantidade em estoque e valor total investido. Agregação por fornecedor e produto.

SQL File 1*

```sql
-- Criação da View do Requisito Q016 - O sistema deve permitir consultar o estoque agrupado por fornecedor,
-- Exibindo nome do fornecedor, categoria, quantidade em estoque e valor total investido.
-- Agregação por fornecedor e produto.
DROP VIEW IF EXISTS vw_q016_estoque_por_fornecedor;
CREATE VIEW vw_q016_estoque_por_fornecedor AS
SELECT
  f.idfornecedor,
  f.nome AS fornecedor,
  p.idproduto,
  p.nome_produto,
  cat.idcategoria,
  cat.nome AS categoria,
  e.qtd_atual,
  e.qtd_minima,
  (e.qtd_atual * p.preco_unitario) AS valor_total_investido
FROM fornecedor f
JOIN fornecedor_produto fp ON fp.fornecedor_idfornecedor = f.idfornecedor
JOIN produto p ON p.idproduto = fp.produto_idproduto
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria
LEFT JOIN estoque e ON e.idproduto = p.estoque_idproduto
ORDER BY f.nome, p.nome_produto;
-- Exemplo:
SELECT * FROM vw_q016_estoque_por_fornecedor WHERE idfornecedor;
```

Result Grid

idfornecedor	fornecedor	idproduto	nome_produto	idcategoria	categoria	qtd_atual	qtd_minima	valor_total_investido
44	Açúcares e Farinhas	17	Feijão Carioca 1kg	4	Mercearia Básica	600	120	5100.00
25	Água Mineral da Fonte	24	Água Mineral sem Gás 1,5L	5	Bebidas	600	120	1500.00
11	Alimentos Frescos SC	1	Banana Nanica (Kg)	1	Hortifrúti	150	30	898.50
11	Alimentos Frescos SC	3	Batata Inglesa (Kg)	1	Hortifrúti	180	40	1420.20
38	Alvejantes Profissionais	37	Água Sanitária 1L	8	Limpeza Doméstica	500	100	2450.00
33	Aves Premium	12	Peito de Frango (Kg)	3	Carnes e Aves	320	60	6048.00
30	Balas e Chicletes Vitoria	49	Chiclete Bola Tutti Frutti	10	Doces e Guloseimas	700	140	350.00
49	Biscoitos Salgados SP	48	Biscoito Recheado Chocolate	10	Doces e Guloseimas	400	80	1280.00
39	Bolos Prontos Delícia	42	Bolo de Fubá Caseiro (Unidade)	9	Padaria	100	20	1500.00

vw_q016_estoque_por_fornece...

Figura 18 - Consulta o estoque agrupado por fornecedor exibindo nome do fornecedor, categoria, quantidade em estoque e valor total investido. Agregação por fornecedor e produto.

17. Criação da View do Requisito Q017 - O sistema deve gerar informações do histórico de compras de um cliente, incluindo produtos adquiridos, quantidades, valores e período. Detalha todas as linhas de itens de venda por cliente.

SQL File 1*

```sql
-- Criação da View do Requisito  Q017 - O sistema deve gerar informações do histórico de compras de um cliente,
-- incluindo produtos adquiridos, quantidades, valores e período.
-- Detalha todas as linhas de itens de venda por cliente.
DROP VIEW IF EXISTS vw_q017_historico_compras_cliente;
CREATE VIEW vw_q017_historico_compras_cliente AS
SELECT
  c.idcliente,
  c.nome AS cliente,
  v.idvenda,
  v.data_venda,
  iv.iditem_venda,
  p.idproduto,
  p.nome_produto,
  iv.quantidade,
  iv.preco_unitario,
  (iv.quantidade * iv.preco_unitario) AS subtotal_item
FROM cliente c
JOIN venda v ON v.cliente_idcliente = c.idcliente
JOIN item_venda iv ON iv.venda_idvenda = v.idvenda
JOIN produto p ON p.idproduto = iv.produto_idproduto
ORDER BY c.idcliente, v.data_venda, v.idvenda;
-- Exemplo:
SELECT * FROM vw_q017_historico_compras_cliente WHERE idcliente AND data_venda BETWEEN '2025-01-01' AND '2025-12-31';
```

Result Grid

idcliente	cliente	idvenda	data_venda	iditem_venda	idproduto	nome_produto	quantidade	preco_unitario	subtotal_item
1	Maria de Fátima Silva	1	2025-01-05	2	16	Arroz Agulhinha 5kg	1	24.99	24.99
1	Maria de Fátima Silva	1	2025-01-05	3	32	Sabonete em Barra Pack 3x90g	1	5.99	5.99
1	Maria de Fátima Silva	1	2025-01-05	1	6	Leite Integral 1L	2	4.59	9.18
1	Maria de Fátima Silva	13	2025-02-03	31	3	Batata Inglesa (Kg)	1	7.89	7.89
1	Maria de Fátima Silva	36	2025-06-01	70	37	Água Sanitária 1L	5	4.90	24.50
2	João Pedro Santos	4	2025-01-07	9	41	Pão Francês (Unidade)	5	0.80	4.00
2	João Pedro Santos	4	2025-01-07	10	33	Creme Dental 90g	2	4.50	9.00
2	João Pedro Santos	47	2025-08-10	85	16	Arroz Agulhinha 5kg	1	24.99	24.99
3	Ana Paula Costa	8	2025-01-11	20	18	Açúcar Refinado 1kg	5	4.20	21.00

vw_q017_historico_compras_di...

Figura 19 - Gerar informações do histórico de compras de um cliente, incluindo produtos adquiridos, quantidades, valores e período. Detalha todas as linhas de itens de venda por cliente.

18. Criação da View do Requisito Q018 - O sistema deve registrar o funcionário responsável por cada venda e permitir a consulta do desempenho por funcionário, incluindo quantidade de vendas, valor total e produtos mais vendidos. Desempenho agregado por funcionário (número de vendas, faturamento).

SQL File 1*

```sql
-- Criação da View do Requisito  Q018 - O sistema deve registrar o funcionário responsável por cada venda e
-- permitir a consulta do desempenho por funcionário, incluindo quantidade de vendas, valor total e produtos mais vendidos.
-- Desempenho agregado por funcionário (número de vendas, faturamento).
DROP VIEW IF EXISTS vw_q018_desempenho_funcionario;
CREATE VIEW vw_q018_desempenho_funcionario AS
SELECT
  f.idfuncionario,
  f.nome AS funcionario,
  f.cargo,
  COUNT(DISTINCT v.idvenda) AS numero_vendas,
  SUM(COALESCE(v.valor_total,0)) AS faturamento_total,
  AVG(COALESCE(v.valor_total,0)) AS ticket_medio
FROM funcionario f
LEFT JOIN venda v ON v.funcionario_idfuncionario = f.idfuncionario
GROUP BY f.idfuncionario, f.nome, f.cargo
ORDER BY faturamento_total DESC;
-- Exemplo:
SELECT * FROM vw_q018_desempenho_funcionario;
```

Result Grid

idfuncionario	funcionario	cargo	numero_vendas	faturamento_total	ticket_medio
4	Jéssica Soares Reis	Caixa - Vespertino	25	1883.19	75.327600
3	Marcelo Aguiar Neto	Caixa - Matutino	25	1375.74	55.029600
1	Pedro Henrique Lima	Gerente Geral	0	0.00	0.000000
2	Camila Viana Paz	Subgerente	0	0.00	0.000000
5	Lucas Barreto Matos	Repositor - Mercearia	0	0.00	0.000000
6	Mariana Fontes Melo	Repositor - Hortifrúti	0	0.00	0.000000
7	Felipe Dias Alves	Repositor - Limpeza	0	0.00	0.000000
8	Larissa Ramos Sales	Açougueiro	0	0.00	0.000000
9	Gustavo Neves Moura	Padeiro	0	0.00	0.000000
10	Aline Martins Silva	Atendente	0	0.00	0.000000

Figura 20 - Registro do funcionário responsável por cada venda e permitir a consulta do desempenho por funcionário, incluindo quantidade de vendas, valor total e produtos mais vendidos. Desempenho agregado por funcionário (número de vendas, faturamento).

19. Criação da View do Requisito Q019 - O sistema deve permitir a consulta de produtos que atingiram o nível mínimo de estoque, exibindo dados do produto e do fornecedor para reposição. Semelhante ao Q010, mas incluindo fornecedor(s) (lista de fornecedores).

SQL File 1*

```sql
-- Criação da View do Requisito  Q019 - O sistema deve permitir a consulta de produtos que atingiram o nível mínimo
-- de estoque, exibindo dados do produto e do fornecedor para reposição.
-- Semelhante ao Q010, mas incluindo fornecedor(s) (lista de fornecedores).
DROP VIEW IF EXISTS vw_q019_produtos_para_reposicao_com_fornecedores;
CREATE VIEW vw_q019_produtos_para_reposicao_com_fornecedores AS
SELECT
  p.idproduto,
  p.nome_produto,
  cat.nome AS categoria,
  e.qtd_atual,
  e.qtd_minima,
  (e.qtd_minima - e.qtd_atual) AS qtd_para_repor,
  -- lista concatenada de fornecedores que oferecem o produto (se existir)
  GROUP_CONCAT(DISTINCT CONCAT(f.idfornecedor,':',f.nome) SEPARATOR ' | ') AS fornecedores
FROM produto p
JOIN estoque e ON e.idproduto = p.estoque_idproduto
LEFT JOIN fornecedor_produto fp ON fp.produto_idproduto = p.idproduto
LEFT JOIN fornecedor f ON f.idfornecedor = fp.fornecedor_idfornecedor
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria
WHERE e.qtd_atual <= e.qtd_minima
GROUP BY p.idproduto, p.nome_produto, cat.nome, e.qtd_atual, e.qtd_minima;
-- Exemplo:
SELECT * FROM vw_q019_produtos_para_reposicao_com_fornecedores;
```

Result Grid

idproduto	nome_produto	categoria	qtd_atual	qtd_minima	qtd_para_repor	fornecedores
11	Acém Bovino (Kg)	Carnes e Aves	40	50	10	3:Frigorífico Boi Forte
14	Filé de Tilápia Congelado 500g	Carnes e Aves	1	30	29	23:Peixes e Frutos do Mar
27	Pizza Muçarela Congelada	Congelados	23	25	2	26:Pizzas Congeladas Top

Figura 21 - consulta de produtos que atingiram o nível mínimo de estoque, exibindo dados do produto e do fornecedor para reposição.

20. Criação da View do Requisito Q020 - O sistema deve calcular o valor total de vendas (faturamento) agrupado por categoria de produto. Agrega receita por categoria (usa item venda * prece unitário).

SQL File 1*

```sql
-- Criação da View do Requisito  Q020 - O sistema deve calcular o valor total de vendas (faturamento) agrupado por categoria de produto.
-- Agrega receita por categoria (usa item_venda * preco_unitario).
DROP VIEW IF EXISTS vw_q020_faturamento_por_categoria;
CREATE VIEW vw_q020_faturamento_por_categoria AS
SELECT
  cat.idcategoria,
  cat.nome AS categoria,
  SUM(iv.quantidade * iv.preco_unitario) AS faturamento_total,
  SUM(iv.quantidade) AS unidades_vendidas,
  COUNT(DISTINCT iv.venda_idvenda) AS vendas_distintas
FROM item_venda iv
JOIN produto p ON p.idproduto = iv.produto_idproduto
LEFT JOIN categoria cat ON cat.idcategoria = p.categoria_idcategoria
GROUP BY cat.idcategoria, cat.nome
ORDER BY faturamento_total DESC;
-- Exemplo:
SELECT * FROM vw_q020_faturamento_por_categoria;
```

Result Grid

idcategoria	categoria	faturamento_total	unidades_vendidas	vendas_distintas
3	Carnes e Aves	508.60	22	10
4	Mercearia Básica	347.82	38	11
2	Laticínios e Frios	297.93	26	8
7	Higiene Pessoal	291.79	19	10
6	Congelados	281.61	16	7
8	Limpeza Doméstica	238.77	37	11
10	Doces e Guloseimas	196.10	34	6
5	Bebidas	187.69	54	7
1	Hortifrúti	144.50	21	9
9	Padaria	115.00	22	10

Figura 22 - Cálculo do valor total de vendas (faturamento) agrupado por categoria de produto.

8. CONCLUSÃO

O desenvolvimento do Sistema de Supermercado (v1.0) possibilitou a construção de um banco de dados relacional inicial, estruturado para gerenciar informações essenciais do cotidiano de um supermercado, como clientes, produtos, fornecedores, funcionários, estoque e vendas. Durante o processo, cada etapa do projeto se integrou de forma progressiva, o que contribuiu para manter a coerência do modelo e o alinhamento com os objetivos propostos.

Já na etapa de modelagem conceitual surgiram desafios relevantes. Traduzir um cenário real em entidades, atributos e relacionamentos não é um processo trivial. Em diferentes momentos, foi necessário revisar decisões, ajustar cardinalidades e reavaliar fluxos que, na prática, apresentam maior complexidade do que o previsto inicialmente. Esses ajustes revelaram-se fundamentais para evitar redundâncias e garantir que a estrutura final fosse capaz de sustentar consultas mais completas, consistentes e próximas das demandas reais do sistema.

Na fase lógica e, principalmente, na física, outros obstáculos apareceram. A criação das chaves, restrições e relacionamentos exigiu cuidado redobrado para evitar inconsistências. Em certos pontos, foi preciso retornar ao modelo inicial, revisar escolhas e adaptar a estrutura para garantir integridade e flexibilidade. Esses pequenos “volta e meia” fazem parte do mundo real dos dados e mostraram como cada decisão influencia todo o sistema.

A implementação no SGBD MySQL representou uma etapa significativa do projeto, pois permitiu observar a aplicação prática dos conceitos desenvolvidos nas fases anteriores. O banco de dados foi criado, populado e testado por meio de consultas SQL, elaboradas para simular cenários reais de operação. Algumas dessas consultas demandaram o uso de subconsultas, múltiplas junções e filtros mais complexos, contribuindo para testar ainda mais a modelagem adotada. Durante esse processo, foram identificados pequenos ajustes necessários, como a adequação de tipos de dados e o refinamento da estrutura de determinadas tabelas, a fim de assegurar maior precisão e integridade ao sistema.

A fase final, evidenciou o potencial prático do sistema desenvolvido. Essa aproximação com situações reais permite compreender de que maneira o banco de dados pode apoiar a gestão do supermercado e como ele responde às demandas operacionais típicas do setor. Além disso, os resultados demonstraram que o sistema ainda possui espaço para expansão, característica considerada positiva no contexto de projetos em evolução.

O projeto, em seu estado atual, apresenta uma base inicial. Entretanto, abre caminho para melhorias futuras, como a integração com módulos financeiros, o desenvolvimento de dashboards analíticos, a implementação de gatilhos (triggers) para automatizar processos sensíveis e a criação de um sistema de fidelidade para clientes. Há também potencial para aprimoramentos relacionados ao desempenho, especialmente em cenários de aumento do volume de dados. Dessa forma, o Sistema de Supermercado (v1.0) não apenas atende ao propósito acadêmico proposto, mas também se configura como um ponto de partida promissor para aplicações reais e sistemas mais robustos.

9. REFERÊNCIAS BIBLIOGRÁFICAS

ASSOCIAÇÃO BRASILEIRA DE SUPERMERCADOS. Ranking ABRAS (2025). Disponível em: https://www.abras.com.br/eventos/ranking-abras/2025.

Acesso em: 18 agosto de 2025 às 21:12.

BEZERRA, Eduardo. **Princípios de Análise e Projeto de Sistemas com UML**. São Paulo: Elsevier, 2006.

DATE, C. J. **Introdução a sistemas de bancos de dados**. Rio de Janeiro: GEN LTC, 2004. E-book.

ELMASRI, Ramez; NAVATHE, Shamkant B**. Sistemas de banco de dados**. 7. ed. São Paulo: Pearson, 2018

FREEMAN, Emily. **DevOps para leigos.** Rio de Janeiro Alta Books 2021 1 recurso online (Os primeiros passos para o sucesso!). ISBN 9788550816661.

VANDERPLAS, Jake. **Guia do Python para Data Science: ferramentas essenciais para trabalhar com dados.** Tradução da 2. ed. Rio de Janeiro: Alta Books, 2025. ISBN 9788550821719.